Introduction à la Programmation et l'Algorithmique

CHRISTELLE CAILLOUET – CHRISTELLE.CAILLOUET@UNIV-COTEDAZUR.FR

DINO LOPEZ PACHECO – DINO.LOPEZ@UNIV-COTEDAZUR.FR

Breaking linearity

break

- To exit from the innermost loop or the switch statement

```
int j = 2;
while (j < 10) {
  int i = 2;
  do {
    if ((j % i) == 0) break;
    i++;
  } while (1);
  printf("%d", i);
  j++;
}
```

continue

- Skip remaining code and go to next loop iteration

```
int i = 0;
while (i < 100) {
  i++;
  if ((i % 2) == 0) continue;
  printf("%d",i);
}
// sortie de printf() ??
```

Types (re)definitions

typedef

- Redefine the name of a type
- Easy the writing/reading of code

```
typedef unsigned char BYTE;
typedef int Integer;
typedef double Dtable[10];
...
BYTE b1;
Integer val;
Dtable mydt;
```

```
typedef struct coord {
  int x; int y;
} Point;

Point ori = {0,0};
Point p1;
p1.x = 1;
p1.y = 2;
```

Bitwise operations

Playing with bits

- &
  - Bitwise AND
  - int val = 8 & 15; //8
- |
  - Bitwise OR
  - int val = 8 |1; //9
- ^
  - Bitwise XOR
  - int val = 1^2; //3
- ~
  - Bitwise NOT
  - unsigned char c = (unsigned char) ~0xF0; // 0xF0 = 240, c = 0x0F = 15
- >>
  - Right shift
  - int val = 8 >> 1; // 4
- <<
  - Left shift
  - int val = 2 << 1; // 4

Pointers

Programs and memory

- To execute programs, the computer needs to load them into memory
  - RAM
  - Every instruction, function, variable is stored at a certain memory location
- Direct manipulation of the memory to solve large scale programming problems
  - Create data collections with dynamic growing/shrinking
  - Manipulate variables' values

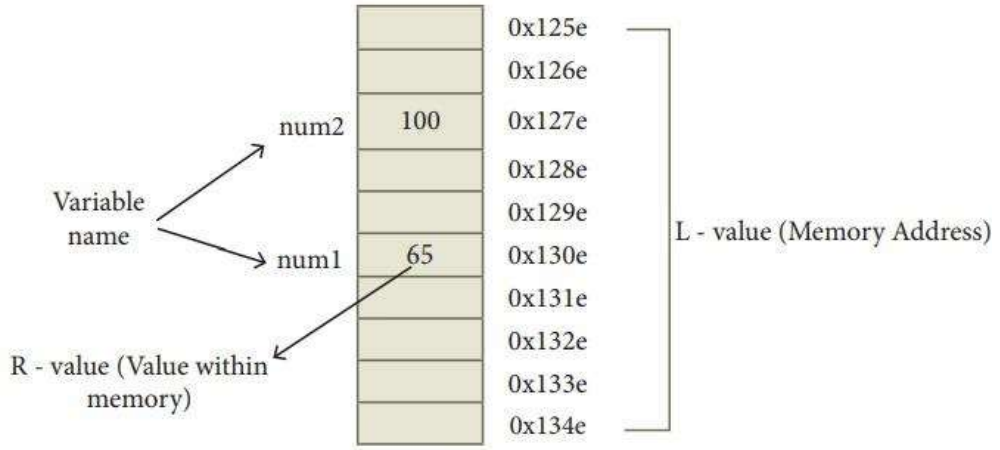


Figure 9.14 Memory allocation of a variable

From https://www.brainkart.com/article/C---program--Variables_36581/

Pointers

- Memory addresses are usually unsigned 32-bits or 64-bit integers
- To manipulate those addresses, C implements the pointer type
- A pointer is a variable whose value is an address
  - Declaration: Use the * notation to indicate that a variable is a pointer

```
int *intp; // the value of pint will be an address storing an int value

double *doublep;

void *vp; // pointer to an address whose data type is currently unknown

char *cp;

int **ipp; // pointer to a pointer whose value address holds one or several int

struct coord *cp; // pointer to a struct coord value type

int *aip[3]; // array of pointers. Every slot in the array will hold an address whose value is an int
```

Assigning to pointer variables

- You need to initialize a pointer variable just like any other variable
  - Initialize to NULL if you don't have a precise address to store yet
  - Initialize to the address of a variable, using the address-of (&) operator
  - Ask for a free memory region (e.g., with malloc()) and store the address of such memory in your pointer

```
struct coord *cp;

cp = NULL;

int var = 10, var2 = 20;

int *pvar = &var;

cp = (struct coord *) malloc(sizeof(struct coord));

// malloc prototype is

// (void *) malloc(size_t numbytes);

int *aip[3];

aip[1] = &var2;
```

Using pointers

- Assigning a pointer value to another pointer

```
int val = 10;

int *pval = &val;

int *p2val = pval; // now, the p2val pointer points to the address of val
```

- Accessing the value of the address stored in a pointer
  - Use the dereference (*) operator

```
printf("%d\n",*p2val); // print 10

(*pval)++; // increase the value of val to 11

*pval++; // Note: increase the address stored at pval (!!) by one unit, then access it content
```

- Print a pointer
  - Use the %p tag
  - Best practice: cast the pointer to a void pointer

```
printf("%p" (void *)pval);
```

Working with struct pointers

- Remember: access the elements of a struct variable with the . operator

```
typedef struct coord {int x; int y;} Coord;

Coord c;

c.x = 0; c.y = 1;

printf("(%d,%d)\n",c.x,c.y);
```

- When having struct pointers, access the elements with the "->" operator

```
Coord *cp;

cp = (Coord *) malloc(sizeof(Coord));

cp->x = 0; cp->y = 0;

printf("(%d,%d)\n",c->x,c->y);
```

Pointer arithmetic

- Compare two pointers with ==, !=, <, >, <=, >=
- Add/subtract an integer from a pointer with +/-
  - If p is a pointer and n the integer to sum, p+n is the nth element after element p
  - If p is a pointer and n the integer to subtract, p-n is the nth element before element p
- Increment/decrement a pointer with ++/--
  - ++ and -- have a higher precedence than the dereference operator *

```
printf("%ld\n",sizeof(int)); // print 4

int arr[5] = {1,3,5,7,9};

int *p = arr;

printf("%p",p); // print 0x8080

p += 2;

printf("%d",*p); // print 5

p--;

printf("%d",*p); // print 3

printf("%p",p); // print 0x8084
```

Pointer arithmetic (cont)

```
char arr[6] = "hello";
char *p = arr;
printf("%p",p); // print 0x8080
p += 2;
printf("%p",p); // print 0x8082 as sizeof(char) == 1
printf("%c",*p); // print l
p--;
printf("%c",*p); // print e
```

Arrays and pointers

- Pointers and arrays behaves very similarly
  - Yet they are not the same!!

```
int ti[5] = {1,3,5,7,9};
printf(“%d”,ti[2]); // put 5 on stdout
int *pi = ti;
printf(“%d”,pi[2]); // put 5 on stdout
int *pi2;
pi2 = malloc(sizeof(int)*5);
// populate pi2 with 0,2,4,6,8
for (int i = 0; i < 5; i++)
   pi2[i] = i*2;
void func(int tab[]);
Void func(int *tab);
```

```
int *pi2;
pi2 = malloc(sizeof(int)*5);
for (int i = 0; i < 5; i++,pi2++)
      *pi2 = i*2;
printf(“%d”,pi[1]); // ??
pi2 -= 5;
printf(“%d”, pi[1]); // ??
// careful, the following doesn’t work
int ti2[5];
ti2 = pi2;
//sizeof(pi) not the same as sizeof(ti)
printf("%d\n",sizeof(pi)); // 8 in a 64-bit architecture
printf("%d\n",sizeof(ti)); // 20, as sizeof(int) gives 4
```

Freeing memory

- When memory is allocated (e.g. using malloc()), it must be freed
  - Non freed allocated memory is lost
  - Memory leak
- Valgrind – memory leak detection tool

```
int *p;
p = malloc(10*sizeof(int));
p = malloc(100*sizeof(int)); // memory leak! We have just lost 10*sizeof(int) bytes!
```

```
int *p;
p = malloc(10*sizeof(int));
free(p)
// previous allocation is freed
// no memory leak so far
p = malloc(100*sizeof(int));
```

Memory allocation in a function

KO

```
int *test(void) {
    int arr[5];
    for (int i=0; i<5; i++)
        arr[i] = i;
    return arr;
}
```

OK

```
int *test(void) {
    int *arr;
    arr = malloc(5*sizeof(int));
    for (int i=0; i<5; i++)
        arr[i] = i;
    // the caller function must free
    // the allocated memory to avoid
    // memory leaks
    return arr;
}
```

Main function prototype

- void main(void)
  - main doesn't return any value, no arguments are expected
  - However, best practice: main() should return zero (success – "exit(0);") or any other value (failure – "exit(-1);")

- int main(int argc, char *argv[])
- int main(int argc, char **argv)
  - Main will return a value and arguments can be passed to the main function program
  - argc = number of arguments
  - argv = array of character pointers
  - You can use any valid variable name
- $ progname arg1 arg2
  - argc → 3
  - argv[0] → "progname"
  - argv1[1] → "arg1"
  - argv1[2] → "arg2"